Design Document

Agentic AI Extension of the Student Survey System
Using LangGraph and FastMCP

Course: SWE 645 — Extra Credit Assignment

Team: Mann Desai | Tisha | Aakash | Yash

Date: April 2026

1. Introduction

For this extra credit assignment, we took the Student Survey system we built in Homework 3 and added an AI-powered layer on top of it. The original system let users submit and manage surveys through a standard React form and a FastAPI REST backend. What we added here is a completely separate conversational interface so that a user can type something like *"delete John Doe's survey"* and the system will figure out what they mean, find the right record, ask for confirmation, and carry out the action — all without the user clicking through any forms.

The core idea was to keep everything we already had intact and build the AI orchestration layer on top of it. We didn't rewrite the backend or touch the existing REST routes; we just exposed the same database operations as MCP tools so the agent could call them.

The overall architecture is shown in Figure 1 below:

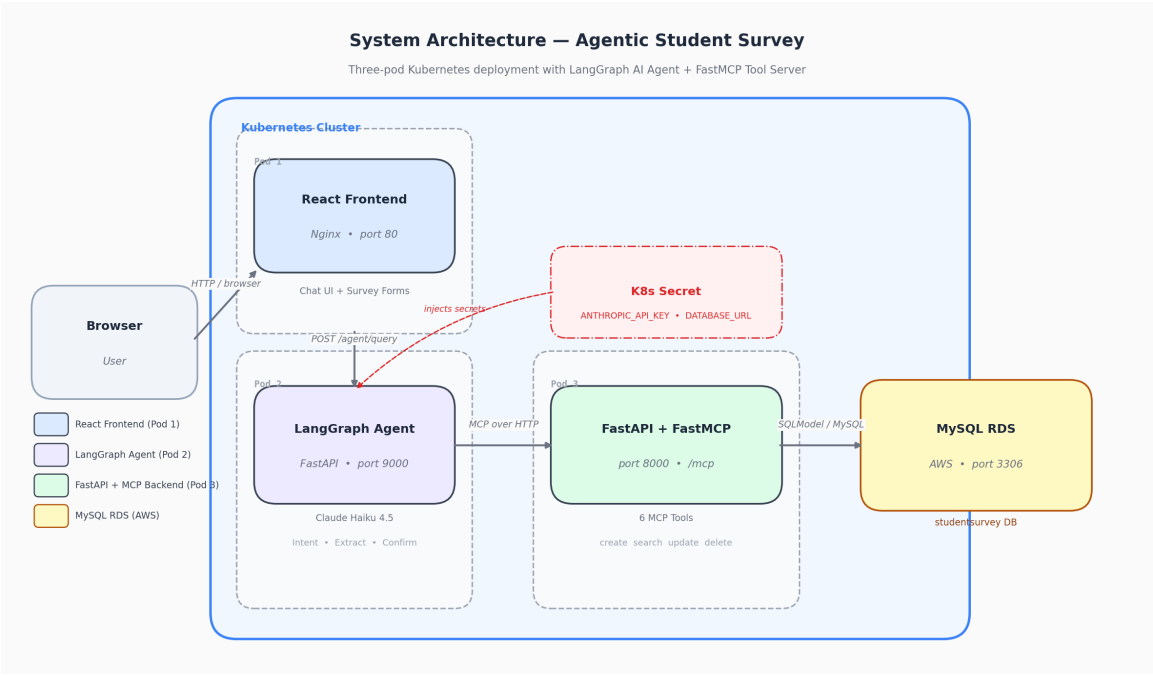

Figure 1 — System Architecture: three Kubernetes pods with LangGraph agent, FastMCP backend, and MySQL RDS.

Each of those three pods is independently deployed on Kubernetes and managed via Helm charts. The agent communicates with the MCP backend over HTTP inside the cluster, and the React frontend talks to the agent through a simple REST endpoint we exposed at `POST /agent/query`.

2. LangGraph Graph Design

The agent is built as a LangGraph `StateGraph`. We chose LangGraph because it lets you define exactly what happens at each step and how decisions are made, rather than relying on the LLM to figure all of that out on its own. For a system that can delete database records, that level of control matters.

State Object

All the information the agent needs across a conversation is stored in a typed dictionary called `AgentState`. The most important fields are:

Field	What it holds
`messages`	Full chat history so the LLM has context
`intent`	What the user wants: create / read / update / delete / confirm / cancel
`draft_survey`	Partially filled survey during a multi-turn create flow
`missing_fields`	Which fields the agent still needs to ask about
`target_survey_id`	ID of the survey being updated or deleted

Field	What it holds
`update_changes`	Specific fields to change in an update
`awaiting`	Whether we're waiting for user confirmation before executing
`response_text`	The final message to send back to the user

The `awaiting` field is what makes the human-in-the-loop safety work. When the agent identifies a survey to delete and displays its details, it sets `awaiting = "delete_confirm"` and returns to the user. On the next message, the router checks that field before doing anything — if the user said "yes," it routes to `delete_execute`; otherwise it cancels.

Graph Topology

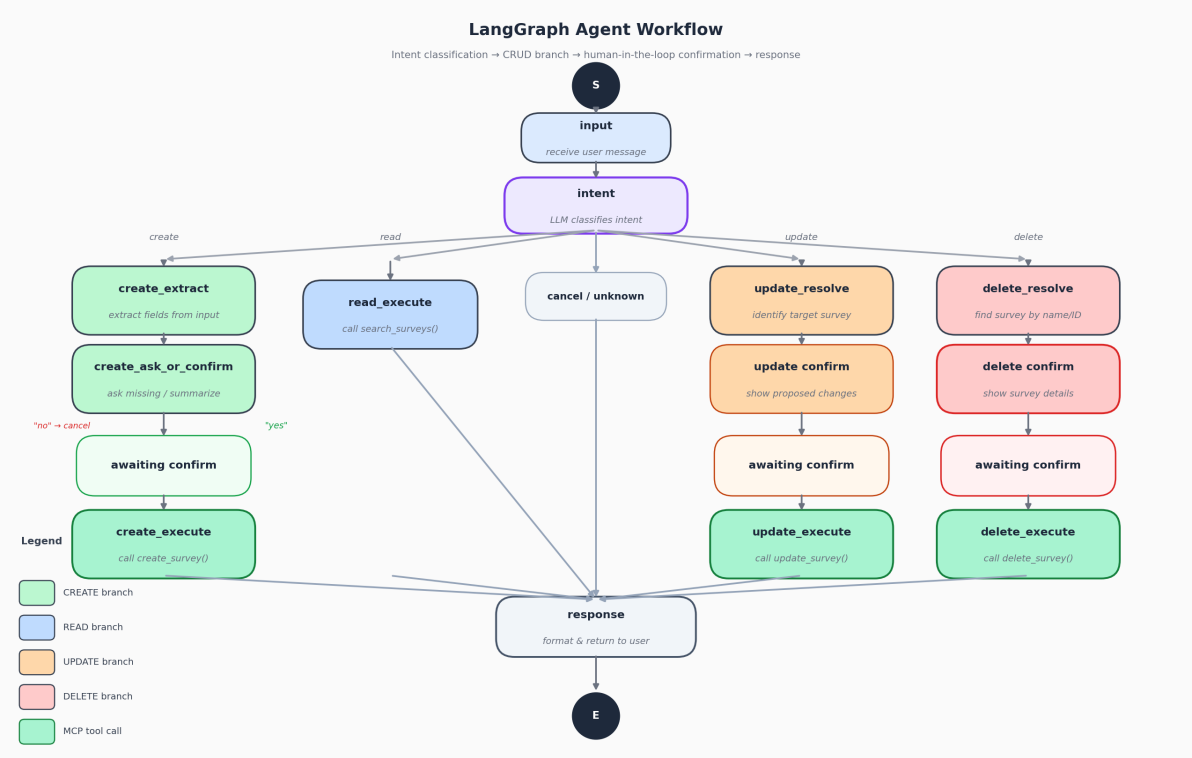

Figure 2 — LangGraph workflow: intent classification routes into four CRUD branches, each with a confirmation gate before destructive operations.

How Routing Works

After the intent node runs, a conditional edge function called `_route_from_intent` decides where to go next. It first checks whether `awaiting` is set — if the user just replied "yes" to a pending delete confirmation, it routes straight to `delete_execute` regardless of how the LLM interpreted the word "yes." That check takes priority over everything else. Only if there's no pending confirmation does it route based on the classified intent.

Session Persistence

LangGraph graphs don't carry state between HTTP requests by themselves. We handle this with an in-memory Python dictionary keyed by `session_id`. Every time the user sends a message, the server rehydrates the stored state, runs the graph, and saves the updated state back. It's simple and works well for a per-session conversation without needing a full external state store.

3. MCP Tools Implemented

The tool layer lives in `backend/mcp_tools.py` and is served by FastMCP, which is mounted as a sub-app inside our existing FastAPI backend at the path `/mcp`. The agent connects to it using `langchain-mcp-adapters`. We implemented all six tools the assignment required:

Tool	Parameters	What it does
`create_survey`	All 12 required fields	Inserts a new survey and returns it with its assigned ID
`list_surveys`	None	Returns every survey record in the database
`get_survey_by_id`	`survey_id`	Fetches a single survey by its numeric ID
`search_surveys`	9 optional filters	Case-insensitive partial-match search by name, city, state, liked_most, interested_via, recommendation, and date range
`update_survey`	survey_id + any subset of fields	Patches only the provided fields, leaves others unchanged
`delete_survey`	`survey_id`	Deletes the survey and returns a snapshot of what was deleted

All tools reuse the same SQLModel ORM session that the REST API uses, so there's one source of truth for the database schema. Each tool has a docstring describing its behavior — this matters because the LLM uses those descriptions to decide which tool to call.

4. React Chat Interface

We added a new page called "AI Survey Assistant" to the existing React app, accessible from a new nav link. The layout follows the suggested design from the assignment spec, with three panels:

Left panel — explains what the agent can do, lists all the fields needed to create a survey, and includes eight clickable example prompts users can try without having to type anything.

Main chat area — a scrollable message thread with user messages on the right (dark blue bubbles) and agent responses on the left (white bubbles). It shows a "Thinking..." indicator while waiting for the agent and supports Enter-to-send.

Right panel — a small log of recent agent actions (created, updated, deleted, queried) so the user can track what's been done without scrolling back through the conversation.

The component generates a UUID session ID when it first loads and sends it with every message. This is how the server knows which in-memory state to load for that conversation. Multi-turn state — like

remembering we're halfway through collecting fields for a new survey — is handled entirely on the server side. The frontend just sends messages and renders responses.

5. Kubernetes and Helm Deployment

We created a Helm chart at `helm/student-survey/` that deploys the full system as three separate pods, matching exactly what the assignment requires:

Pod	Image	Service Type
Frontend (React + Nginx)	`student-survey-frontend`	LoadBalancer — port 80
Agent (LangGraph)	`student-survey-agent`	ClusterIP — port 9000
Backend (FastAPI + MCP)	`student-survey-backend`	ClusterIP — port 8000

Sensitive values — the Anthropic API key and the database connection string — are stored as Kubernetes Secrets and injected as environment variables at pod startup. No credentials are hardcoded anywhere in the Helm chart templates. The `values.yaml` has placeholder fields for base64-encoded secrets that get passed in at deploy time via `--set`.

The agent deployment reads `MCP_SERVER_URL` pointing to the backend service's cluster-internal DNS name, so all communication between the agent and the backend stays inside the cluster. Only the frontend is exposed externally through the LoadBalancer.

6. Challenges We Ran Into

MCP Tool Result Format

This one took a while to figure out. Depending on the version of `langchain-mcp-adapters`, the same tool call could return a plain Python dict, a JSON string, or a list of MCP content blocks that looked like `[{"type": "text", "text": "..."}]`. The agent nodes were breaking because they expected a dict but kept getting a list. We ended up writing a small `_unwrap_tool_result()` utility that normalizes all three formats before any node tries to read from the result.

Keeping Confirmation State Across HTTP Requests

The trickiest part of the whole project was making multi-turn confirmation work reliably. When the agent says "I found John Doe's survey. Do you want to delete it?" and the user replies "yes," the server receives a brand new HTTP request with no built-in memory. If we just ran intent classification on "yes," it would come back as `confirm` — but the router also needs to know it's confirming a *delete*, not a create or update. The fix was to save the `awaiting` field in the session store and check it before routing, so the router always knows what confirmation is pending.

FastMCP Lifespan Inside FastAPI

FastMCP requires its own lifespan context to initialize the MCP session manager. When we mounted it as a sub-app inside FastAPI, the session manager wasn't starting and every tool call was failing with a connection error. The fix was to nest the MCP lifespan inside our FastAPI lifespan using `asynccontextmanager`, so both start up together. Once we figured that out it worked perfectly, but it wasn't obvious from the FastMCP docs.

Intent Classification for Short Replies

Early on, when a user typed "yes" after a delete confirmation prompt, the LLM would sometimes classify it as `unknown` or even `create` depending on the conversation context. We fixed this by adding `confirm` and `cancel` as explicit intent classes in the classification prompt with worked examples, and by always routing based on `awaiting` first before trusting the classifier.

Vite Environment Variables at Build Time

We wanted to pass the agent URL into the React app via Docker Compose, but Vite bakes `VITE_*` variables at image build time, not at container runtime. So environment variables set in `docker-compose.yaml` under the frontend service don't actually get picked up. We resolved this by setting safe fallback values in the component code (`http://localhost:8000` and `http://localhost:9000`), which work correctly when running locally since those ports are exposed on the host.

7. Tech Stack Summary

Layer	Technology
Frontend	React 19, React Router, Axios, Vite, Nginx
AI Agent	LangGraph, LangChain, Claude Haiku 4.5 (Anthropic), FastAPI
Tool Server	FastMCP 2.10+, langchain-mcp-adapters
Backend ORM	SQLModel (SQLAlchemy + Pydantic)
Database	MySQL 8 on AWS RDS
Containers	Docker, Docker Compose
Orchestration	Kubernetes, Helm 3